
Pipeline Simplifié d'un Encodeur Vidéo de type MPEG-4

Redigé par :

Hamouti Nour El Malek *Groupe 3*

Tifour Aya *Groupe 2*

Avril 2026

Contents

1	Introduction	2
1.1	Objectifs	2
2	a. Description du pipeline	2
2.1	schéma descriptif	2
2.2	Étape 1 : Pré-traitement	4
2.3	Étape 2 : Codage Intra-frame (I-frames)	4
2.4	Étape 2.1 : Reconstruction au Décodage (<code>decode_iframe</code>)	5
2.5	Étape 3 : Compression Temporelle (P-frames) et Structure GOP	5
2.6	Étape 3.1 : Reconstruction des P-frames (<code>decode_pframe</code>)	6
2.7	Étape 3.2 : Gestion globale du flux (<code>encode_video</code>)	6
2.8	GOP (Group of Pictures)	6
2.9	Étape 4 : Codage Entropique et Sérialisation	6
2.10	Étape 4.1 : Lecture et Décompression (<code>read_bin</code>)	6
2.11	Étape 5 : Évaluation et Visualisation du Pipeline	7
3	Design Choices & Justification	8
3.1	Justification du facteur de quantification (Q_{factor})	9
4	Analyse expérimentale	10
4.1	Taux de compression	10
4.2	Compression en fonction du facteur de quantification (QF)	10
4.3	Impact du GOP	10
4.4	PSNR par frame	11
5	Conclusion	12

1 Introduction

Ce projet consiste à implémenter un pipeline simplifié d'encodage vidéo inspiré du standard MPEG-4. L'objectif est de compresser une séquence d'images et de la reconstruire à partir d'un fichier binaire compressé.

1.1 Objectifs

- Réduire la taille d'une vidéo à partir d'images successives
- Appliquer des techniques de compression spatiale et temporelle
- Reconstituer les images avec une perte contrôlée

2 a. Description du pipeline

2.1 schéma descriptif

Le pipeline est composé de cinq étapes principales :

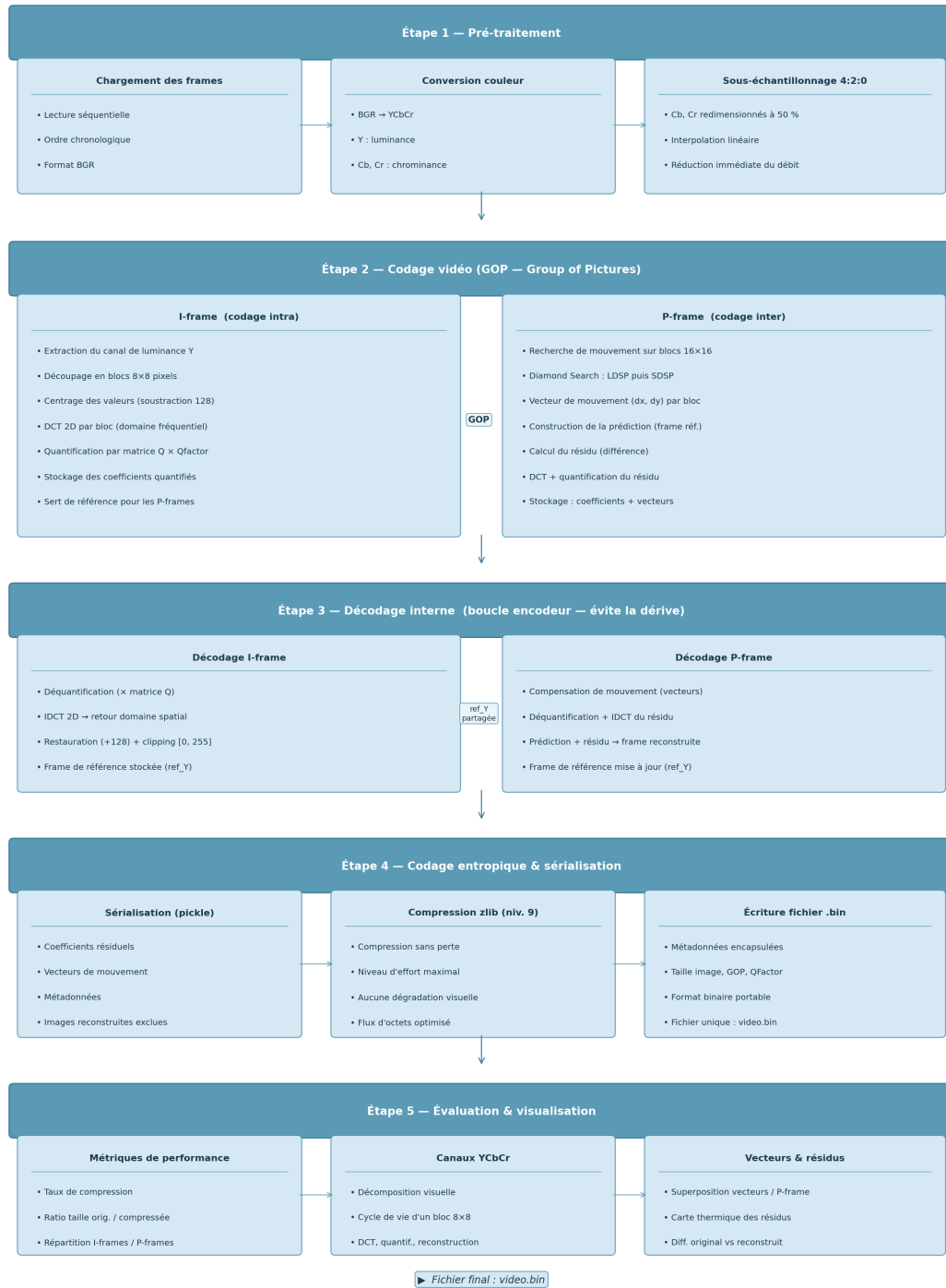


Figure 1: Schéma descriptif du pipeline MJPEG

2.2 Étape 1 : Pré-traitement

La première étape de notre encodeur MPEG-4 consiste à préparer les données brutes pour optimiser la compression spatiale et temporelle ultérieure. Cette phase se décompose en trois fonctions principales :

- **Chargement des données (load_frames)** : Les images séquentielles du dossier source sont chargées en respectant l'ordre chronologique. Chaque image est initialement représentée dans l'espace de couleur BGR (Blue-Green-Red).
- **Conversion d'espace colorimétrique (bgr_to_ycbcr)** : Nous convertissons les images BGR vers l'espace **YCbCr**.
 - **Y** représente la luminance (luminosité).
 - **Cb** et **Cr** représentent la chrominance (teinte bleue et rouge).
- **Sous-échantillonnage de la chrominance (Chroma Subsampling)** : Pour réduire immédiatement la taille des données, nous appliquons un sous-échantillonnage de type **4:2:0**. Les canaux Cb et Cr sont redimensionnés à 50% de leur largeur et hauteur originales à l'aide d'une interpolation linéaire.

2.3 Étape 2 : Codage Intra-frame (I-frames)

La deuxième étape de la pipeline concerne la création des **I-frames** (Intra-coded frames). Ces trames servent de points de référence et sont compressées individuellement, sans exploiter les informations des images précédentes ou suivantes. Le processus repose principalement sur la Transformée en Cosinus Discrète (DCT).

1. **Découpage en blocs et centrage** : Le canal de luminance Y est divisé en blocs de 8×8 pixels. Pour chaque bloc, une opération de centrage est effectuée en soustrayant 128 à la valeur de chaque pixel (amenant la plage $[0, 255]$ vers $[-128, 127]$), ce qui optimise le calcul de la DCT.
2. **Transformée en Cosinus Discrète (dct_2d)** : La DCT est appliquée à chaque bloc de 8×8 . Cette opération mathématique convertit les données spatiales (pixels) en données fréquentielles. Elle permet de concentrer l'énergie visuelle (les informations les plus importantes) dans les coefficients de basse fréquence, situés en haut à gauche de la matrice du bloc.
3. **Quantification (quantize)** : Chaque bloc de coefficients DCT est ensuite divisé par une matrice de quantification standard (**Q_MATRIX**), multipliée par un facteur de qualité (**Q_factor**).
 - Les coefficients de haute fréquence, souvent liés à des détails imperceptibles, sont réduits vers zéro.
 - C'est cette étape de "division et arrondi" qui génère la perte d'information, mais permet une réduction massive du poids des données.

2.4 Étape 2.1 : Reconstruction au Décodage (decode_iframe)

Pour reconstruire l'image, le décodeur effectue les opérations inverses de manière symétrique :

1. **Dé-quantification (dequantize)** : Les coefficients quantifiés sont multipliés par la même matrice Q pour retrouver des valeurs proches des coefficients DCT originaux.
2. **DCT Inverse (idct_2d)** : La transformée inverse repasse les données du domaine fréquentiel au domaine spatial.
3. **Restauration** : On ajoute à nouveau 128 aux valeurs obtenues pour retrouver la plage de couleurs d'origine, puis on sature les valeurs (clipping) entre 0 et 255 pour éviter les erreurs d'affichage.

2.5 Étape 3 : Compression Temporelle (P-frames) et Structure GOP

Cette étape exploite la redondance temporelle entre trames successives pour maximiser le taux de compression. Elle repose sur le concept de *Group of Pictures* (GOP) et l'estimation de mouvement.

1. **Structure du Groupe d'Images (GOP)** : La vidéo est organisée en séquences de taille G . La première trame du groupe est codée en tant qu'**I-frame** (référence complète), tandis que les $G-1$ trames suivantes sont des **P-frames** (Predicted frames), codées par rapport à la trame précédente.
2. **Estimation de Mouvement (motion_estimation)** : Chaque P-frame est divisée en macroblocs de 16×16 pixels. Pour chaque macrobloc, l'encodeur utilise un algorithme **Diamond Search** en deux phases : une recherche grossière (LDSP, pas de 2 pixels) qui itère jusqu'à convergence, suivie d'un raffinement au pixel près (SDSP, pas de 1 pixel). La similarité est mesurée par la MSE :

$$MSE = \frac{1}{N} \sum_{i=0}^{N-1} (C_i - R_i)^2, \quad N = 256$$

où C_i et R_i sont les pixels du bloc courant et de référence. Le déplacement trouvé est enregistré sous forme de **vecteur de mouvement** (dx, dy) , réduisant la complexité de $\mathcal{O}(S^2)$ à $\mathcal{O}(\log S)$.

3. **Calcul du Résidu et Codage (encode_pframe)** : Une fois le vecteur de mouvement identifié, l'encodeur calcule la **prédiction** (le bloc trouvé dans la référence) et le soustrait au bloc actuel pour obtenir le **résidu**. Ce résidu (la différence) est ensuite compressé via le même processus que les I-frames : découpage en sous-blocs 8×8 , application de la DCT et quantification.

2.6 Étape 3.1 : Reconstruction des P-frames (decode_pframe)

Le processus de reconstruction par le décodeur combine les informations spatiales et temporelles :

1. **Compensation de mouvement** : Le décodeur utilise les vecteurs de mouvement pour extraire le bloc de prédiction correspondant dans la trame de référence déjà décodée.
2. **Addition du résidu** : Le résidu est décodé (dé-quantification et IDCT) puis ajouté pixel par pixel à la prédiction pour reconstituer la trame finale.

2.7 Étape 3.2 : Gestion globale du flux (encode_video)

L'encodage complet de la vidéo gère l'alternance entre trames I et P selon le paramètre GOP. L'encodeur doit lui-même décoder chaque trame compressée pour l'utiliser comme référence (`ref_Y`) pour la trame suivante, garantissant ainsi que le décodeur et l'encodeur utilisent exactement la même base visuelle et évitent la dérive (drift).

2.8 GOP (Group of Pictures)

Une image I est insérée tous les G frames, les autres images sont des P-frames.

2.9 Étape 4 : Codage Entropique et Sérialisation

Le codage entropique constitue la dernière étape de l'encodage, visant une compression sans perte des données déjà traitées par DCT et quantification.

- **Sérialisation (pickle)** : Toutes les structures de données Python (coefficients résiduels, vecteurs de mouvement et métadonnées) sont converties en un flux d'octets. Les images reconstruites sont exclues de ce processus pour ne conserver que le strict nécessaire au décodeur.
- **Compression sans perte (zlib)** : Ce flux d'octets subit une compression via l'algorithme `zlib` avec un niveau d'effort maximal (niveau 9). Cette étape réduit la taille du fichier final sans aucune dégradation supplémentaire de la qualité visuelle.
- **Écriture du fichier (write_bin)** : Les données compressées sont encapsulées avec les métadonnées (taille des images, GOP, facteur de quantification) dans un fichier binaire unique d'extension `.bin`.

2.10 Étape 4.1 : Lecture et Décompression (read_bin)

Le processus inverse est appliqué lors du décodage : le fichier est lu, décompressé par `zlib`, puis désérialisé pour restaurer les structures de données originales nécessaires à la reconstruction de la vidéo.

2.11 Étape 5 : Évaluation et Visualisation du Pipeline

Cette phase finale permet de quantifier les performances de l'encodeur et de rendre compte visuellement du traitement subi par les données.

- **Calcul des Métriques** : Le système compare la taille brute des images originales (calculée en octets selon la résolution) avec la taille réelle du fichier binaire compressé. Le ratio de compression et la répartition des types de trames (I vs P) sont extraits.
- **Tableau de Bord de Visualisation** : Un outil basé sur `matplotlib` a été développé pour générer une figure synthétique regroupant :
 1. La décomposition des canaux YCbCr.
 2. Le cycle de vie d'un bloc 8×8 (DCT, quantification et reconstruction).
 3. La superposition des vecteurs de mouvement sur les trames P.
 4. La carte thermique des résidus (différence entre l'original et le reconstruit).

La figure suivante illustre la visualisation complète des différentes étapes du pipeline:

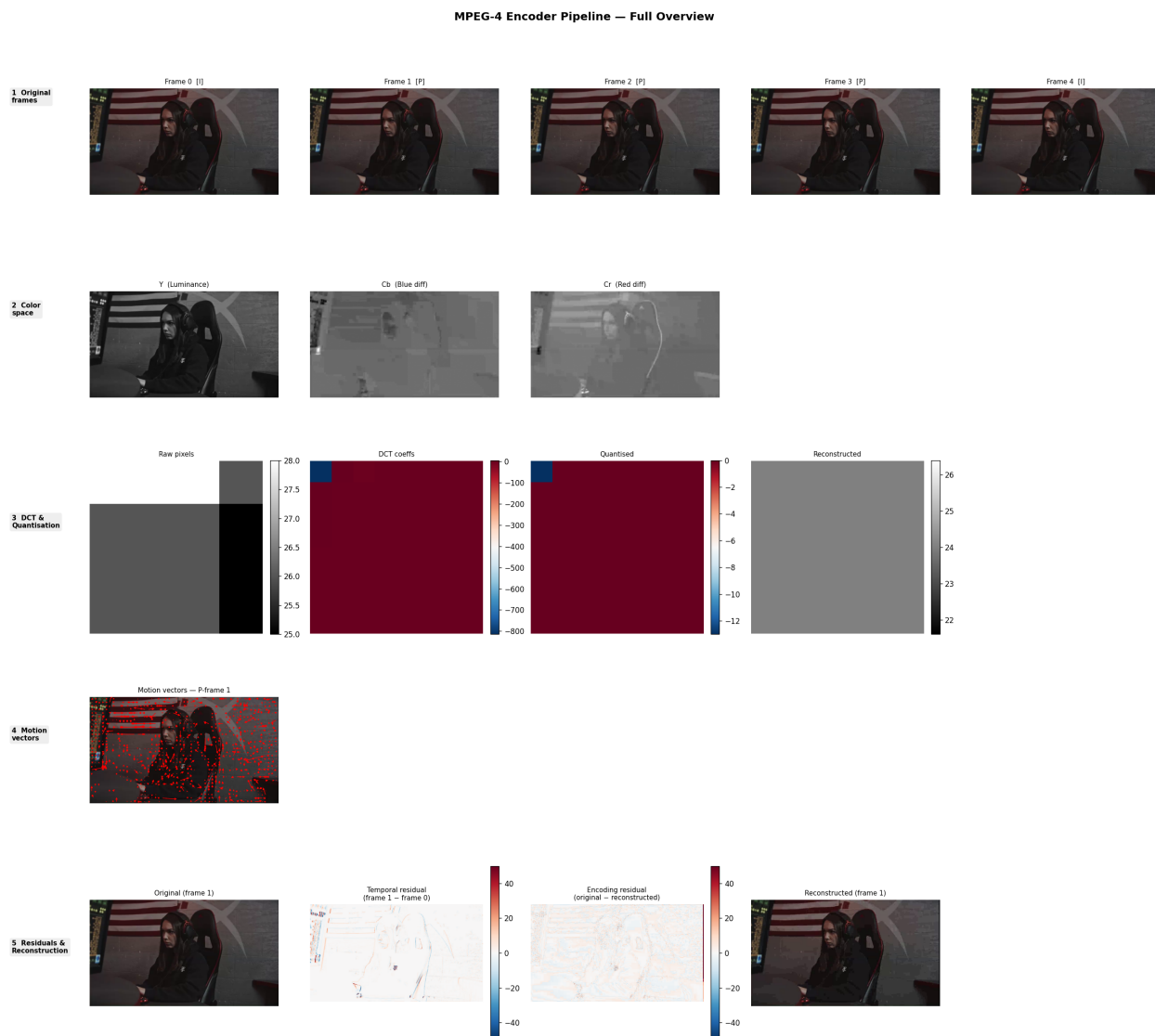


Figure 2: Visualisation complète du pipeline de compression et reconstruction vidéo.

3 Design Choices & Justification

Le tableau suivant récapitule les choix techniques effectués lors de l'implémentation de l'encodeur et les justifications associées en termes de compression et de qualité.

Étape du Pipeline	Choix de Conception	Justification Technique
1. Pré-traitement	Passage au format YCbCr 4:2:0	Décorrélation de la luminance et de la chrominance. Le sous-échantillonnage réduit les données de 50% avec un impact visuel minime (sensibilité de l'œil).
2. Codage Intra	DCT par blocs 8×8	Standard de l'industrie (JPEG/MPEG). Permet de passer du domaine spatial au domaine fréquentiel pour isoler les détails superflus.
2. Codage Intra	Matrice de Quantification standard	Équilibre optimal entre la suppression des hautes fréquences (compression) et la conservation de la structure de l'image.
3. Codage Inter	Macroblocs 16×16	Compromis entre précision du mouvement et coût de calcul/stockage des vecteurs de mouvement.
3. Codage Inter	Mesure de similarité par MSE	L'Erreur Quadratique Moyenne est simple à calculer et reflète bien la différence d'énergie entre deux blocs pour minimiser le résidu.
3. Codage Inter	Structure GOP (Group of Pictures)	Évite l'accumulation d'erreurs (drift) en insérant des images de référence (I-frame) de manière périodique.
4. Codage Entropique	Combinaison Pickle + Zlib (level 9)	Pickle assure la flexibilité des structures de données (dictionnaires/listes), tandis que Zlib garantit une compression sans perte maximale sur le flux binaire.

Table 1: Synthèse des choix de conception et justifications.

3.1 Justification du facteur de quantification (Q_{factor})

L'ajout d'un multiplicateur Q_{factor} sur la matrice de quantification est un choix de conception majeur. Il permet à l'utilisateur de piloter dynamiquement le curseur entre :

- **Débit (Bitrate)** : Un Q_{factor} élevé augmente le nombre de zéros après quantification, rendant le codage entropique beaucoup plus efficace.
- **Distorsion** : Un Q_{factor} faible préserve les hautes fréquences de la DCT, garantissant une reconstruction plus fidèle à l'original.

4 Analyse expérimentale

4.1 Taux de compression

$$\text{Taux de compression} = \frac{\text{Taille originale}}{\text{Taille compressée}} \quad (1)$$

4.2 Compression en fonction du facteur de quantification (QF)

Plus le QF augmente, plus la compression augmente. Mais à partir de QF=2, le gain devient très faible (88.68 → 91.06 pour QF=4). Par contre, un QF trop grand dégrade la qualité de l'image (effet de blocs).

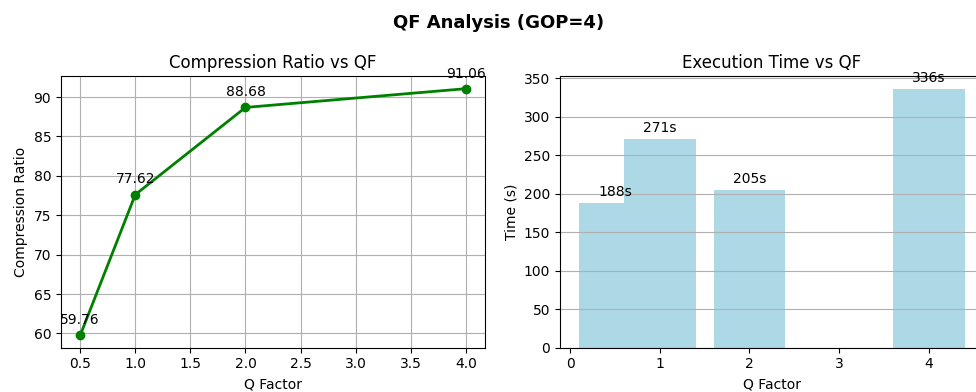


Figure 3: Taux de compression en fonction du facteur de quantification

4.3 Impact du GOP

Plus le GOP est grand, meilleure est la compression (de 50.05 pour GOP=2 jusqu'à 170.04 pour GOP=8), car on a plus de P-frames qui sont plus légères que les I-frames. Le temps d'exécution par contre n'est pas stable, car il dépend du contenu de la vidéo et pas seulement du GOP.

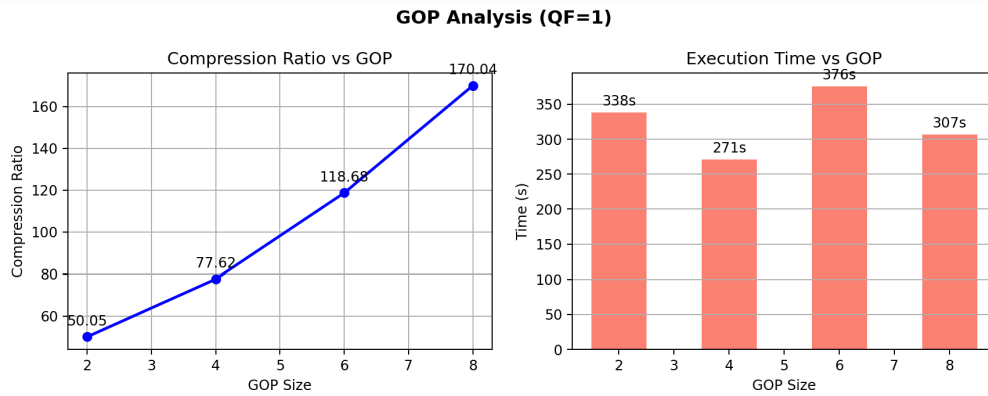


Figure 4: Impact du GOP sur la compression

Le meilleur choix reste **GOP=4 et QF=1** : bonne compression, bon temps d'exécution, et qualité acceptable.

4.4 PSNR par frame

Le PSNR (Peak Signal-to-Noise Ratio) mesure la fidélité de chaque frame reconstruite par rapport à l'originale. Il est calculé sur le canal Y (luminance) selon :

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \quad (2)$$

où la MSE est la moyenne des erreurs au carré pixel par pixel.

Le graphique ci-dessous montre le PSNR pour chacune des 300 frames (bleu = I-frame, orange = P-frame). Le PSNR moyen obtenu est de **32.90 dB**, ce qui correspond à une qualité de reconstruction acceptable. On observe que les I-frames présentent généralement un PSNR plus élevé que les P-frames, car elles sont codées sans référence et accumulent moins d'erreurs de prédiction.

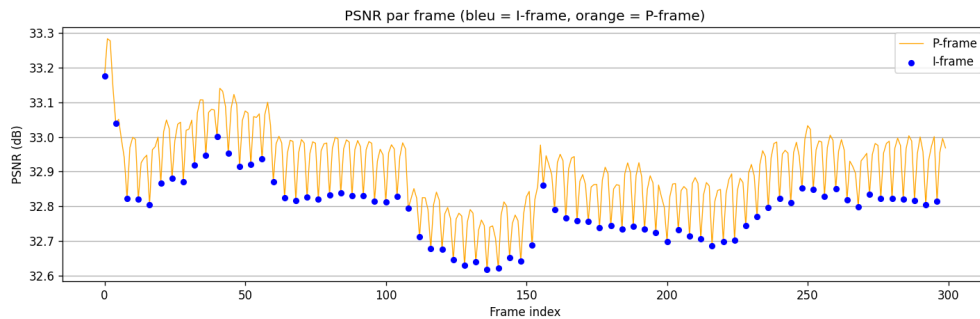


Figure 5: PSNR par frame (bleu = I-frame, orange = P-frame)

5 Conclusion

Ce projet a permis de simuler un pipeline simplifié d'encodage vidéo inspiré de MPEG-4, combinant compression spatiale (DCT + quantification) et temporelle (estimation de mouvement, GOP). Le fichier binaire final de 2863.9 KB regroupe l'ensemble des coefficients et vecteurs de mouvement compressés sans perte via zlib. Les métriques PSNR calculées par frame confirment une qualité de reconstruction acceptable, avec un PSNR moyen de 32.90 dB. Les principales limites résident dans l'absence d'un codage entropique pur (Huffman, RLE) et le calcul limité au canal Y pour les métriques. Des pistes d'amélioration incluent l'ajout des B-frames, un codage en zig-zag des coefficients DCT, et le calcul du PSNR sur les 3 canaux YCbCr.